CRBench: A Method-Agnostic, Resource-Aware Evaluation Framework for Long-Context Large Language Models

Om Chimurkar*

Newton School of Technology, Rishihood University

om.chimurkar2024@nst.rishihood.edu.in · omchimurkar45@gmail.com

https://github.com/Omc12/CRBench

Research Preprint — Version 0.2.0 (Research Preview)
August 2026

Abstract

As Large Language Models (LLMs) expand their context windows to tens and hundreds of thousands of tokens, the Key-Value (KV) cache becomes the primary operational bottleneck during autoregressive inference, consuming tens of gigabytes of GPU memory and severely limiting batch concurrency. While dozens of context compression paradigms have emerged—spanning KV quantization, attention-sink eviction, semantic token merging, low-rank projection, and latent state representations—existing evaluation methodology remains fragmented. Current long-context benchmarks assess downstream task accuracy in isolation without measuring memory consumption, while systems papers report compression ratios on unstandardized tasks without an anchored baseline.

In this work, we present **CRBench** (*Context Resource Benchmark*), a method-agnostic, query-level evaluation framework that quantifies how much contextual capability an LLM retains relative to the memory and system resources it consumes. CRBench defines the atomic evaluation primitive as the 4-tuple $(\mathcal{M}, q, \mathcal{R}_{\text{dense}}, \mathcal{A})$, evaluating each query pairwise against the base model's uncompressed FP16/BF16 dense reference. We define a reference-anchored quality retention metric Q, an analytical and physical resource efficiency metric R_{mem}, and a resource-aware Part 1 utility score $\mathcal{S}_{\text{res}} = \alpha Q + (1-\alpha) R_{\text{mem}}$ (default $\alpha = 0.70$). We further specify a decoupled Part 2 system track incorporating Time-To-First-Token (TTFT), prefill latency, and decode throughput. We describe the complete architecture of CRBench v0.2.0, release an open-source extensible framework supporting custom adapters in under 50 lines of code, evaluate scoring desiderata across seven candidate formulations, and report preliminary validation on a 1.5B-parameter open-weight model across five representative tasks. This preprint establishes the formal methodology, architectural specifications, and empirical protocol of CRBench v0.2.0, preceding a planned journal submission with expanded larger model families and extended context horizons up to 128K tokens.

Keywords: Long-Context LLMs, KV Cache Compression, Resource-Aware Evaluation, Memory Efficiency, Benchmarking, Inference Systems.

Status Note (Research Preprint v0.2.0): This manuscript is an academic preprint establishing the theoretical formulation, software architecture, query-level methodology, and initial small-model empirical validation of CRBench v0.2.0, preceding an expanded journal submission with comprehensive benchmarks across larger model architectures and extended context horizons ($>$ 32K up to 128K tokens).

*Newton School of Technology, Rishihood University. Correspondence: om.chimurkar2024@nst.rishihood.edu.in, omchimurkar45@gmail.com

1 Introduction

Modern Large Language Models (LLMs) have achieved remarkable breakthroughs across multi-document synthesis, extended multi-hop question answering, repository-scale code analysis, and complex agentic workflows [1, 3, 6, 23]. These capabilities rely on long-context processing, expanding model context windows from 2,048 tokens to 32,768, 128,000, and even 1,000,000 tokens [9, 21].

However, this contextual expansion is fundamentally bounded by the memory footprint of the Key-Value (KV) cache in standard Transformer architectures [24]. Because standard self-attention caches intermediate key and value tensor representations for every token across all attention heads and layers, KV cache memory scales linearly with sequence length L and batch size B:

$$\boxed{M_{\text{KV}} = 2 \times N_{\text{layers}} \times N_{\text{kv\_heads}} \times d_{\text{head}} \times L \times B \times \text{sizeof(dtype)} \quad \text{[bytes]}} \tag{1}$$

For a representative 8-billion parameter model (e.g., Llama-3.1-8B [12] with 32 layers, 8 Key-Value heads with Grouped-Query Attention, and head dimension 128 in 16-bit precision), a single batch-1 128K-token context sequence ($L = 131{,}072$) requires exactly 16.0 GiB (17.18 GB) of memory purely for the KV cache ($2 \times 32 \times 8 \times 128 \times 131{,}072 \times 2$ bytes)—exceeding the model's static 16-bit parameter footprint ($\approx$ 15.0 GiB / 16.1 GB) and exhausting the VRAM of standard consumer GPUs.

To alleviate this severe bottleneck, the research community has proposed a rich taxonomy of KV cache compression techniques:

1. **KV Cache Quantization**: Representing keys and values in low-precision numerical formats, such as FP8, INT8, INT4, or non-uniform 2-bit formats with per-channel or group-wise scaling [10, 15, 20, 27].

2. **KV Cache Eviction & Sparsification**: Dynamically discarding unimportant historical tokens based on attention score accumulation, sliding local windows, or attention sinks (e.g., StreamingLLM [25], SnapKV [18], H_2O [26], Scissorhands [19]).

3. **KV Cache Merging & Clustering**: Combining spatially or semantically adjacent token vectors through temporal pooling, hierarchical clustering, or weighted centroids [13, 29].

4. **Low-Rank & Subspace Projection**: Projecting KV tensors into low-dimensional latent subspaces via principal component analysis, singular value decomposition, or learned linear projections [7, 22].

5. **Dynamic & Disentangled KV (DKV) Representations**: Factoring context memory into shared static subspace bases and sparse dynamic token updates.

Despite this rapid algorithmic progress, an acute methodological gap exists: *the field lacks a unified, method-agnostic, resource-aware evaluation benchmark.* Current long-context evaluation suites (e.g., Needle-In-A-Haystack [17], RULER [16], LongBench [4], LongBench v2 [5]) are primarily designed to evaluate model capability under uncompressed dense representations rather than to standardize quality retention under varying memory constraints. Conversely, papers introducing compression algorithms report accuracy on idiosyncratic task subsets alongside theoretical compression ratios, without controlling for base model capability, metadata storage overheads, or query-level variance.

To resolve these challenges, we introduce **CRBench** (*Context Resource Benchmark*), an open, extensible, and principled evaluation framework designed to quantify how much contextual capability an LLM retains for the memory and system resources it consumes. While long-context LLM applications motivate sequence horizons spanning 128K to 1M tokens, this initial

v0.2.0 preprint validates the software architecture, query-level execution pipeline, and multi-paradigm adapters on 2K–4K token sequence lengths on an open-weight 1.5B foundation model, establishing a verified functional foundation preceding planned larger-scale evaluations.

CRBench is *not* an empirical bake-off designed to crown a single compression technique. Rather, it is a reusable benchmarking harness and scoring specification that researchers and practitioners can apply to evaluate any existing or novel context memory representation.

Method Neutrality & Authorship Disclosure. Differential KV (DKV) [8] was introduced by the primary author. To maintain strict evaluation neutrality and prevent benchmarking bias, CRBench is designed as a method-agnostic, open-source framework with standardized, identical execution pipelines across all candidate adapters, fully public query-level harness code, and transparent reproducibility across all 10 evaluated configurations.

Core Contributions. The primary contributions of this work are:

- **The Query-Level Evaluation Primitive**: We formulate benchmark evaluation around the atomic 4-tuple $(\mathcal{M}, q, \mathcal{R}_{\text{dense}}, \mathcal{A})$, pairing every compressed method evaluation directly against the identical model's uncompressed dense FP16 reference on the exact same prompt, reducing prompt-selection and base-model capability confounding.
- **Model-Relative Capability Normalization**: We specify a reference-anchored quality metric $Q \in [0, 100]$ that isolates contextual representation fidelity from the base model's intrinsic zero-shot strength, incorporating a dynamic range gate to prevent division instabilities on small or saturated models.
- **Explicit Analytical & Physical Memory Accounting**: We formalize effective bits-per-element (b_{eff}) and memory efficiency R_{mem}, accounting not only for raw tensor payloads but also quantization scales, codebooks, eviction index bitmaps, and alignment overheads.
- **Desiderata-Based Scoring Utility Analysis**: We evaluate seven candidate utility formulations against five key measurement desiderata (quality monotonicity, resource monotonicity, boundedness, Pareto consistency, and low-quality penalty criteria), proposing the linear additive formulation $\mathcal{S}_{\text{res}} = \alpha Q + (1 - \alpha) R_{\text{mem}}$ (default $\alpha = 0.70$) as a transparent, non-destructive scoring formulation.
- **Two-Track Benchmark Separation**: We decouple Part 1 (algorithmic context representation efficiency) from Part 2 (hardware-dependent system latency, TTFT, and decode throughput), enabling pure theoretical comparisons alongside deployability profiles.
- **Open-Source Extensible Implementation**: We release the modular CRBench v0.2.0 Python package featuring custom adapter registration in under 50 lines of code, non-destructive score recomputation, transparent failure categorization, and preliminary validation on an open-weight 1.5B model.

2 Motivation & Related Work

2.1 Limitations of Current Evaluation Paradigms

Existing approaches to evaluating long-context models and compression techniques suffer from three structural shortcomings, summarized in Table 1:

1. **Quality–Resource Disconnection**: Traditional benchmarks (e.g., L-Eval [2], BAMBOO [11], InfiniteBench [28]) measure accuracy without tracking memory. A method that achieves 90% accuracy while consuming 16 bits per token is treated identically to a method achieving 90% accuracy at 4 bits per token, despite the latter offering 4× greater hardware efficiency.

Table 1: **Comparison of Common Evaluation Paradigms and Capabilities Provided by CRBench.** Summary of dimensions captured across long-context accuracy benchmarks, synthetic needle tests, inference profilers, and CRBench.

Evaluation Paradigm	Quality	Memory	System Latency	Dense-Anchored	Query-Level Pairwise
Standard Long-Context (e.g., LongBench)	✓	×	×	×	×
Synthetic Needle Tests (e.g., NIAH, RULER)	✓	×	×	×	×
Inference Systems Profilers (e.g., vLLM)	×	✓	✓	×	×
Ad-hoc Compression Papers	✓	△	△	×	×
CRBench (This Work)	✓	✓	✓	✓	✓

2. **Confounding Base Model Capability with Representation Quality**: Reporting absolute task accuracy confuses the model's fundamental reasoning capability with the efficiency of the KV compression algorithm. A weak compression technique applied to a larger foundation model might achieve a higher absolute accuracy than an advanced compression method on a compact model. By anchoring evaluations to each model's uncompressed dense execution on the same query, CRBench isolates representation efficiency.

3. **Ignoring Metadata and System Overhead**: Many compression works claim theoretical 4-bit or 2-bit compression while omitting the memory required for scaling factors, outlier vectors, or eviction index tables. Furthermore, algorithms that save memory by executing uncoalesced memory reads or expensive CPU–GPU synchronization can severely degrade prefill latency (TTFT) and decode throughput.

To address KV memory growth, KVQuant [15], KIVI [20], and QA-LoRA [14] introduced per-channel and asymmetric 2-to-4-bit quantization. Eviction policies such as StreamingLLM [25] demonstrated the critical importance of initial "attention sinks" combined with local rolling buffers. SnapKV [18] and H_2O [26] select heavy-hitter key tokens during prefill. PyramidKV [29] allocates layer-wise dynamic budgets. Despite their individual merits, each paper evaluates on disparate task sets, distinct token budgets, and incomparable metrics. CRBench provides the common benchmarking infrastructure required to evaluate these techniques uniformly.

3 Problem Definition & Mathematical Formulation

Figure 1 illustrates the overall architecture of CRBench, depicting the dual execution branches, reference-anchored normalization, and decoupled scoring tracks.

3.1 The Atomic Evaluation Primitive

Let $\mathcal{M}$ denote an autoregressive language model parameterized by weights Θ. An evaluation dataset $\mathcal{D} = \{(x_i, y_i, \tau_i, L_i)\}_{i=1}^{N}$ consists of queries x_i, ground-truth target outputs y_i, task types τ_i, and prompt context token lengths L_i.

Let $\mathcal{A}$ denote a candidate context memory adapter (e.g., INT4 quantization or SnapKV eviction) operating under an assigned resource budget $\mathcal{B}$. In standard dense inference, the uncompressed reference adapter is denoted $\mathcal{R}_{\text{dense}}$ (16-bit FP16/BF16 representation).

Definition 1 (CRBench Atomic Evaluation Unit)**.** *The fundamental evaluation unit in CRBench is the 4-tuple:*

$$\boxed{\mathcal{U}_i = \Big(\mathcal{M},\ (x_i, y_i),\ \mathcal{R}_{dense},\ \mathcal{A}(\mathcal{B})\Big)} \tag{2}$$

For every query (x_i, y_i), CRBench executes:

1. ***Dense Baseline Execution**: Model $\mathcal{M}$ generates prediction $\hat{y}_{i,dense} = \mathcal{M}(x_i \mid \mathcal{R}_{dense})$.*

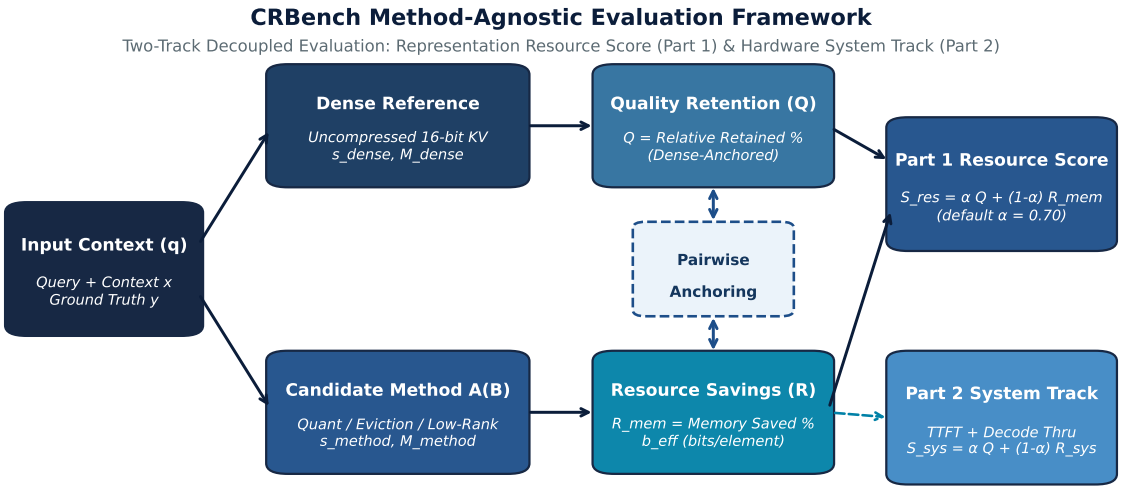

Figure 1: **CRBench Method-Agnostic Evaluation Framework Architecture.** The benchmark decouples evaluation into two complementary tracks: Part 1 (Algorithmic Representation Efficiency $\mathcal{S}_{\text{res}}$) and Part 2 (Hardware System Track $\mathcal{S}_{\text{sys}}$). Every candidate context representation $\mathcal{A}(\mathcal{B})$ is evaluated pairwise against the base model's uncompressed dense reference $\mathcal{R}_{\text{dense}}$ on the identical input query (x_i, y_i).

2. ***Candidate Adapter Execution***: *Model $\mathcal{M}$ generates prediction $\hat{y}_{i,method} = \mathcal{M}(x_i \mid \mathcal{A}(\mathcal{B}))$.*
3. ***Quality Measurement***: *Raw task scores $s_{i,dense} = Eval(\hat{y}_{i,dense}, y_i) \in [0,1]$ and $s_{i,method} = Eval(\hat{y}_{i,method}, y_i) \in [0,1]$ are recorded.*
4. ***Resource Measurement***: *Memory footprints $M_{i,dense}$ and $M_{i,method}$ (in bytes) are computed analytically and tracked physically.*

3.2 Model-Relative Quality Retention (Q)

To decouple adapter evaluation from the base model's intrinsic zero-shot capability, CRBench measures *quality retention relative to the model's uncompressed dense performance on that specific query*:

Definition 2 (Normalized Quality Retention)**.** *Let $s_{floor}(\tau)$ denote the empirical random-chance performance floor for task τ (e.g., 0.0 for open retrieval, 0.25 for 4-choice QA). The normalized capability retention $Q_i \in [0.0, 100.0]$ is defined as:*

$$\boxed{Q_i = \min\left(100.0,\ \max\left(0.0,\ \frac{s_{i,method} - s_{floor}}{\max\left(\Delta_{\min},\ s_{i,dense} - s_{floor}\right)} \times 100.0\right)\right)} \tag{3}$$

where $\Delta_{\min} = 0.05$ (5%) is a dynamic range regularizer. Retention Q_i is explicitly capped at 100.0, ensuring that if a compressed method matches or slightly exceeds the uncompressed reference on an individual prompt, capability retention does not exceed 100%.

Dynamic Range Gating. If $s_{i,\text{dense}} - s_{\text{floor}} < \Delta_{\min}$, the base model itself cannot perform the task even with full, uncompressed 16-bit KV memory. In this regime, attempting to evaluate retention ratio produces extreme numerical noise. When $s_{i,\text{dense}} - s_{\text{floor}} < \Delta_{\min}$, CRBench sets $Q_i = 0.0$ (unless $s_{i,\text{method}} > s_{i,\text{dense}}$, in which case $Q_i = \min(100.0, \frac{s_{i,\text{method}} - s_{\text{floor}}}{\Delta_{\min}} \times 100)$).

3.3 Memory Resource Accounting (R_{mem})

CRBench enforces explicit state memory accounting. For any adapter $\mathcal{A}$, the total memory consumed by the context representation is partitioned into three additive components:

$$\boxed{M_{\text{method}}(L) = M_{\text{algo}}(L) + M_{\text{meta}}(L) + M_{\text{align}}(L) \quad [\text{bytes}]} \tag{4}$$

where:

- M_{algo} is the raw compressed tensor payload (e.g., 4 bits per parameter $\times$ stored token count).
- M_{meta} is all mandatory auxiliary metadata, including FP16 per-channel quantization scale factors, zero-point offsets, codebook dictionaries, or 32-bit integer token position index arrays for eviction methods.
- M_{align} represents hardware memory alignment and padding overheads.

Definition 3 (Effective Bits Per KV Element)**.** *The effective bits-per-element (b_{eff}) measures the total memory footprint normalized by the total uncompressed Key-Value tensor scalar count ($2 \times N_{layers} \times N_{kv\_heads} \times d_{head} \times L$):*

$$\boxed{b_{eff} = \frac{8 \cdot M_{method}(L)}{2 \cdot N_{layers} \cdot N_{kv\_heads} \cdot d_{head} \cdot L} \quad [bits/element]} \tag{5}$$

For standard uncompressed 16-bit float KV caches (FP16/BF16), $b_{eff} = 16.0$ bits/element. The corresponding context-normalized memory footprint is $b_{eff,tok} = \frac{8 \cdot M_{method}(L)}{L}$ bits/token.

Definition 4 (Memory Resource Savings)**.** *The normalized memory resource savings $R_{mem,i} \in [0.0, 100.0]$ is:*

$$\boxed{R_{mem,i} = 100.0 \times \max\left(0.0,\ 1.0 - \frac{M_{i,method}}{M_{i,dense}}\right)} \tag{6}$$

4 CRBench Scoring Formulation

4.1 Measurement Desiderata for Utility Formulation

A scientifically rigorous resource benchmark score $\mathcal{S}(Q, R)$ mapping quality retention $Q \in [0, 100]$ and resource savings $R \in [0, 100]$ to a scalar metric $\mathcal{S} \in [0, 100]$ is evaluated against five key measurement desiderata, with the systematic audit of candidate formulations summarized in Table 2:

1. **Desideratum 1 (Quality Monotonicity)**: $\frac{\partial \mathcal{S}}{\partial Q} > 0$. For any fixed resource savings, strictly higher quality retention yields a strictly higher benchmark score.
2. **Desideratum 2 (Resource Monotonicity)**: $\frac{\partial \mathcal{S}}{\partial R} > 0$. For any fixed non-zero quality retention, greater memory savings yields a strictly higher benchmark score.
3. **Desideratum 3 (Boundedness)**: $\mathcal{S}(Q, R) \in [0.0, 100.0]$ for all $(Q, R) \in [0, 100]^2$, with $\mathcal{S}(0, 0) = 0.0$ and $\mathcal{S}(100, 100) = 100.0$.
4. **Desideratum 4 (Pareto Dominance Consistency)**: If method $\mathcal{A}$ dominates method $\mathcal{B}$ (i.e., $Q_A \geq Q_B$ and $R_A \geq R_B$ with at least one strict inequality), then $\mathcal{S}(Q_A, R_A) > \mathcal{S}(Q_B, R_B)$.
5. **Desideratum 5 (Low-Quality Penalty Criterion)**: Under the benchmark's default quality-dominant configuration, a method with catastrophic quality degradation (e.g., $Q < 10\%$) does not outscore the uncompressed dense baseline ($Q = 100\%, R = 0\%$) simply through aggressive memory compression ($R \to 100\%$).

Table 2: **Evaluation of Candidate Scoring Formulations Against Measurement Desiderata.** Seven mathematical formulations were systematically evaluated across $Q, R \in [0, 100]$. The linear formulation satisfies the specified desiderata under the benchmark's default configuration and tested ranges.

ID	Mathematical Formula	Q-Mono	R-Mono	Bounded	Pareto Consistency	Low-Q Penalty
F1	$\mathcal{S} = \alpha Q + (1-\alpha)R$ **[CRBench Part 1]**	✓	✓	✓	**100% (36/36)**	✓
F2	$\mathcal{S} = Q^{\alpha} R^{1-\alpha}$ (Cobb-Douglas)	✓	✓	✓	100% (36/36)	✓*
F3	$\mathcal{S} = \left(\frac{\alpha}{Q} + \frac{1-\alpha}{R}\right)^{-1}$ (Harmonic)	✓	✓	✓	100% (36/36)	✓*
F4	$\mathcal{S} = \left(\alpha Q^2 + (1-\alpha)R^2\right)^{0.5}$ (Power Mean $p = 2$)	✓	✓	✓	100% (36/36)	×
F5	$\mathcal{S} = Q \cdot (R/100)^{\alpha}$	✓	✓	✓	100% (36/36)	✓*
F6	$\mathcal{S} = \min(Q^{\alpha}, R^{1-\alpha})$	×	×	✓	83% (30/36)	✓
F7	$\mathcal{S} = \frac{Q}{100} \cdot (\alpha Q + (1-\alpha)R)$ (Gated Linear)	✓	✓	✓	100% (36/36)	✓

*Note: Multiplicative and harmonic formulations assign $\mathcal{S} = 0.0$ whenever $R = 0\%$. In CRBench, we explicitly prioritize anchoring the uncompressed baseline to a non-zero reference utility reflecting its full contextual capability ($\mathcal{S}_{\text{res}} = 70.0$), rather than assigning zero utility to uncompressed representations.

4.2 Part 1: Resource Utility Score ($\mathcal{S}_{\text{res}}$)

To address these desiderata transparently without introducing opaque nonlinear distortion, CRBench defines the canonical Part 1 Resource Score using a linear additive utility formulation:

$$\boxed{\mathcal{S}_{\text{res},i} = \alpha \cdot Q_i + (1-\alpha) \cdot R_{\text{mem},i}} \tag{7}$$

where $\alpha \in [0.10, 0.90]$ is the global quality preference weight, with standard configuration $\alpha = 0.70$.

Interpretation of Default $\alpha = 0.70$. The current $\alpha = 0.70$ configuration reflects a quality-dominant design choice prioritizing contextual reasoning fidelity (70%) over pure compression ratio (30%); its optimal setting across diverse foundation models and workloads remains subject to ongoing empirical validation. Under this configuration:

- The uncompressed dense reference ($Q = 100\%, R = 0\%$) receives a baseline score of $\mathcal{S}_{\text{res}} = 0.70 \times 100 + 0.30 \times 0 = 70.0$.
- A high-fidelity 4-bit method retaining 94% quality with 75% memory savings achieves $\mathcal{S}_{\text{res}} = 0.70 \times 94 + 0.30 \times 75 = 88.30$, accurately recognized as superior resource utility compared to the dense reference.
- A degraded 2-bit method retaining only 10% quality despite 87.5% memory savings achieves $\mathcal{S}_{\text{res}} = 0.70 \times 10 + 0.30 \times 87.5 = 33.25$, correctly penalized far below the dense baseline.

As shown in Figure 2(a), nonlinear formulations such as Cobb-Douglas and Harmonic Mean assign zero utility to uncompressed baselines ($R = 0\%$) despite perfect quality retention. In CRBench, we adopt the linear formulation because it provides transparent, continuous trade-off dynamics while preserving an interpretable non-zero score for the uncompressed dense reference. In Figure 2(b), we sweep α across the full valid domain $\alpha \in [0.10, 0.90]$, demonstrating that relative method ordering remains strictly invariant across the realistic quality-dominant operational band $\alpha \in [0.40, 0.85]$, with $\alpha = 0.70$ serving as the default operating point. At extreme memory-dominant weights ($\alpha < 0.40$), aggressive eviction methods overtake high-fidelity representations as memory savings outweigh contextual fidelity.

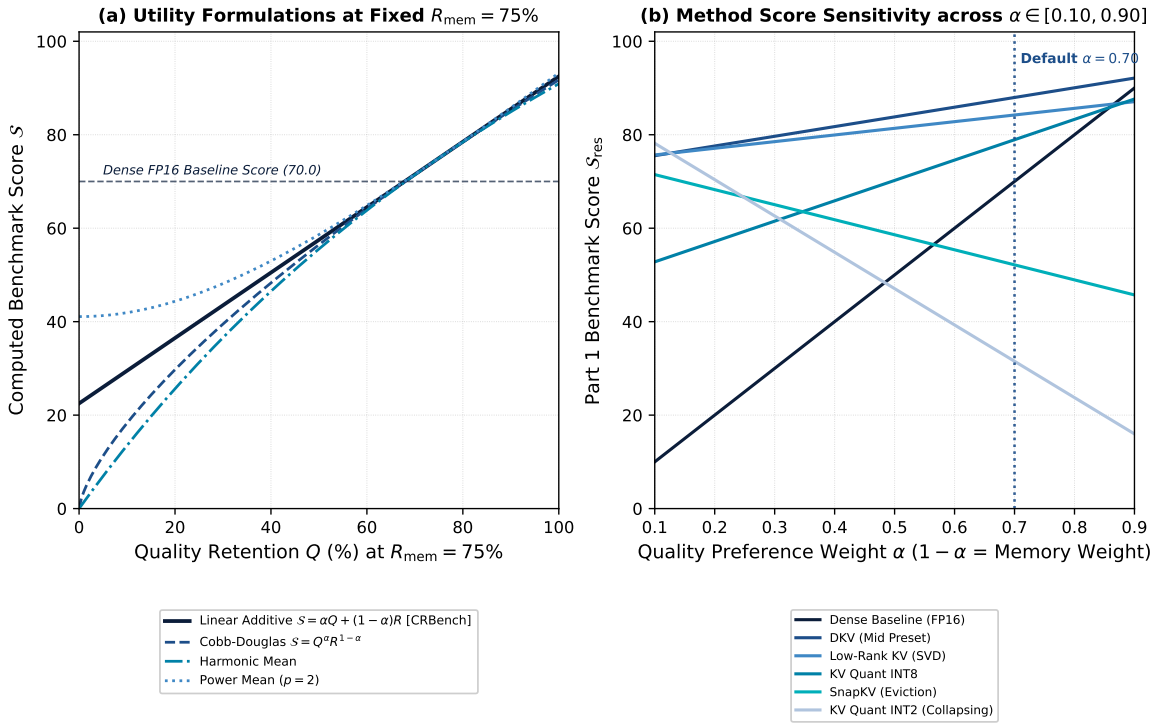

Figure 2: **Mathematical Behavior and Sensitivity Analysis of Scoring Utility Formulations.** (a) Score trajectory $\mathcal{S}(Q, R)$ across quality retention $Q \in [0, 100]\%$ at fixed memory savings $R_{\text{mem}} = 75\%$, contrasting the linear additive formulation against Cobb-Douglas, Harmonic, and Power Mean alternatives. (b) Sensitivity of candidate method scores across preference weight $\alpha \in [0.10, 0.90]$, illustrating ranking stability on the preliminary 1.5B evaluation set across $\alpha \in [0.40, 0.85]$ and highlighting the default $\alpha = 0.70$ quality-dominant operating point.

4.3 Dataset, Depth Integration (AUQC), and Multi-Context Aggregation

For an evaluation dataset $\mathcal{D}$ containing $|\mathcal{D}|$ queries, the aggregate benchmark score is computed by averaging query-level utility scores:

$$\boxed{\bar{\mathcal{S}}_{\text{res}} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \mathcal{S}_{\text{res},i}, \quad \bar{Q} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} Q_i, \quad \bar{R}_{\text{mem}} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} R_{\text{mem},i}} \tag{8}$$

To ensure statistical rigor, CRBench computes 95% non-parametric bootstrap confidence intervals $\text{CI}_{95}(\bar{\mathcal{S}}_{\text{res}})$ with $B = 2,000$ resamples (for example, on our preliminary 1.5B evaluation, DKV Mid achieves $\bar{\mathcal{S}}_{\text{res}} = 88.0$ with $\text{CI}_{95} = [87.1, 88.9]$ and Low-Rank KV achieves 84.2 with $\text{CI}_{95} = [83.3, 85.1]$, showing clear separation in the preliminary bootstrap intervals).

Area Under the Quality Curve (AUQC). For contextual retrieval and tracking tasks evaluated across parameterized needle insertion depths $D = \{d_1, d_2, \ldots, d_m\} \subset [0, 1]$ at context length L, we define the Area Under the Quality Curve (AUQC) as the depth-integrated contextual retention score:

$$\boxed{\text{AUQC}(L) = \frac{1}{|D|} \sum_{d \in D} Q(d, L) \in [0, 100]} \tag{9}$$

where $Q(d, L)$ represents normalized quality retention for a query with evidence placed at relative depth d.

Multi-Context Length Weighting. When evaluating a suite spanning multiple context lengths $\mathcal{L} = \{L_1, L_2, \ldots, L_K\}$ (e.g., 2K, 4K, 8K, 16K, 32K), CRBench assigns normalized logarithmic context weights:

$$\boxed{w_L = \frac{1 + \log_2(L/L_{\min})}{\sum_{L' \in \mathcal{L}} \left(1 + \log_2(L'/L_{\min})\right)}} \tag{10}$$

Logarithmic weighting reflects the geometric difficulty of context scaling while preventing ultra-long sequences from totally eclipsing intermediate lengths. CRBench also supports uniform ($w_L = 1/|\mathcal{L}|$) and linear ($w_L \propto L$) schemes.

5 System Evaluation Track (Part 2)

While Part 1 evaluates algorithmic memory representation efficiency, practical deployment requires analyzing physical latency and throughput. An adapter that compresses tensors effectively but relies on un-fused Python loops or irregular memory gathers may suffer severe latency degradation.

CRBench Part 2 introduces a dedicated *System Evaluation Track* that integrates runtime latency profiling while remaining conceptually decoupled from Part 1.

5.1 Measured System Profiling Metrics

CRBench incorporates hardware-synchronized wall-clock timers and device memory allocators to capture:

1. **Time-To-First-Token (TTFT / Prefill Latency, TTFT)**: Total wall-clock time (in milliseconds) required to process the prompt context of length L and emit the first token.

2. **Autoregressive Decode Throughput ($\mathcal{T}_{\text{dec}}$)**: Generation speed measured in tokens per second (tok/s) during subsequent autoregressive token emission:

$$\boxed{\mathcal{T}_{\text{dec}} = \frac{N_{\text{generated}}}{\Delta t_{\text{decode}}} \quad [\text{tok/s}]} \tag{11}$$

3. **Peak Device VRAM ($\mathcal{M}_{\text{peak}}$)**: Maximum allocated and reserved GPU memory during inference.

5.2 Part 2 System Efficiency (R_{sys}) and Score ($\mathcal{S}_{\text{sys}}$)

To combine memory savings with runtime speedup without distorting the algorithmic Part 1 score, CRBench defines a composite system efficiency metric $R_{\text{sys}} \in [0.0, 100.0]$:

$$\boxed{R_{\text{sys},i} = w_{\text{mem}} \cdot R_{\text{mem},i} + w_{\text{ttft}} \cdot R_{\text{ttft},i} + w_{\text{thru}} \cdot R_{\text{thru},i}} \tag{12}$$

where $w_{\text{mem}} = 0.50, w_{\text{ttft}} = 0.25, w_{\text{thru}} = 0.25$, and the normalized runtime efficiency components are defined relative to the dense baseline:

$$\boxed{R_{\text{ttft},i} = \min\left(100.0,\ 50.0 \times \frac{\text{TTFT}_{i,\text{dense}}}{\text{TTFT}_{i,\text{method}}}\right)}, \quad \boxed{R_{\text{thru},i} = \min\left(100.0,\ 50.0 \times \frac{\mathcal{T}_{i,\text{method}}}{\mathcal{T}_{i,\text{dense}}}\right)}. \tag{13}$$

Under parity ($\text{TTFT}_{\text{method}} = \text{TTFT}_{\text{dense}}$ and $\mathcal{T}_{\text{method}} = \mathcal{T}_{\text{dense}}$), $R_{\text{ttft}} = R_{\text{thru}} = 50.0$. Speedups scale toward 100.0, while slowdowns scale toward 0.0.

We emphasize that the weights ($w_{\text{mem}} = 0.50, w_{\text{ttft}} = 0.25, w_{\text{thru}} = 0.25$) represent a provisional system-track configuration rather than an empirically fixed constant. Part 2 is an

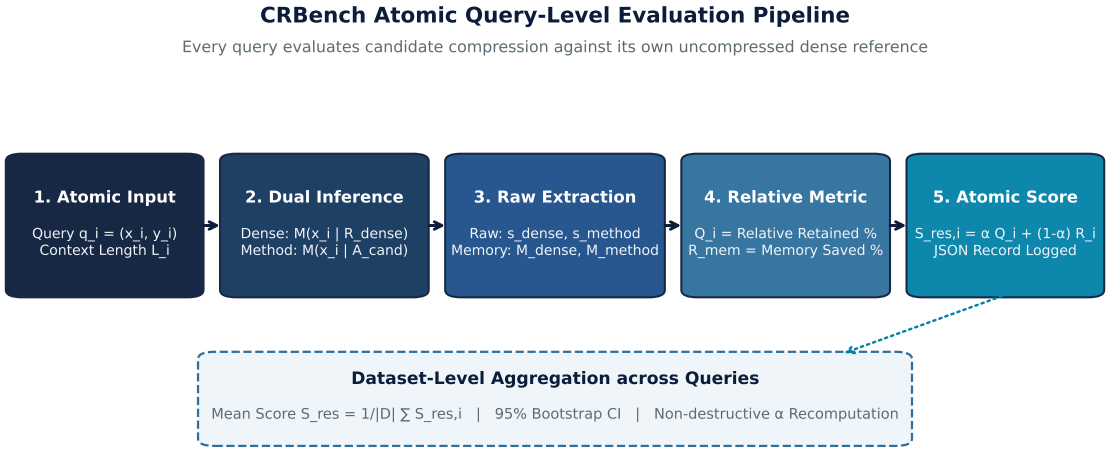

Figure 3: **CRBench Atomic Query-Level Evaluation Pipeline.** Execution flow from single query input through parallel dense and candidate forward passes, exact metric extraction, dynamic range-gated quality normalization, atomic scoring, and dataset-level aggregation.

exploratory track designed to illustrate how system-level trade-offs can be integrated alongside representation quality; these weights will be systematically ablated and refined following broader multi-hardware validation.

The composite **CRBench Part 2 System Score** is then:

$$\boxed{\mathcal{S}_{\text{sys},i} = \alpha \cdot Q_i + (1 - \alpha) \cdot R_{\text{sys},i}} \tag{14}$$

6 Software Architecture & Implementation

CRBench v0.2.0 is engineered as a modular, open-source Python framework designed for reproducibility, extensibility, and hardware portability. Figure 3 visualizes the atomic query execution pipeline.

6.1 Extensible Context Adapter Interface

To evaluate any novel KV representation, researchers implement `BaseContextAdapter` by defining three core methods, executed through the atomic flow detailed in Algorithm 1:

Adapter Implementation in Under 50 Lines. A researcher creating a custom low-rank or dynamic memory adapter only needs to subclass `BaseContextAdapter` and register it via `@Registry.register_adapter("my_method")`. The adapter defines its tensor transformation hooks and reports its explicit storage parameters via `get_kv_metadata()`.

6.2 Non-Destructive Score Recomputation

A central design principle of CRBench is complete separation between expensive model inference and scoring logic. CRBench saves raw token outputs, ground truths, prefill/decode timestamps, and byte allocations into a versioned JSON schema (`raw_results_v1.json`).

Researchers can re-evaluate benchmark scores under alternative α values, alternative context weights, or different utility formulations instantaneously via the CLI without re-executing inference:

```
crbench recompute --raw-file raw_results_v1.json --alpha 0.80 --formula linear
```

Algorithm 1 Atomic Query Evaluation Flow in CRBench

Require: Model $\mathcal{M}$, Query (x, y), Dense Reference $\mathcal{R}_{\text{dense}}$, Adapter $\mathcal{A}$, Budget $\mathcal{B}$, Task Floor s_{floor}

Execute Dense Baseline: $\hat{y}_{\text{dense}}, \text{prof}_{\text{dense}} \leftarrow \text{Generate}(\mathcal{M}, x, \mathcal{R}_{\text{dense}})$
Calculate Raw Dense Score: $s_{\text{dense}} \leftarrow \text{Eval}(\hat{y}_{\text{dense}}, y)$
Apply Budget to Adapter: $\mathcal{A}.\text{apply\_budget}(\mathcal{B}, \text{length}(x))$
Execute Candidate Adapter: $\hat{y}_{\text{method}}, \text{prof}_{\text{method}} \leftarrow \text{Generate}(\mathcal{M}, x, \mathcal{A})$
Calculate Raw Method Score: $s_{\text{method}} \leftarrow \text{Eval}(\hat{y}_{\text{method}}, y)$
Compute Normalized Quality: $Q \leftarrow \min\left(100.0,\ \max\left(0.0,\ \frac{s_{\text{method}} - s_{\text{floor}}}{\max(\Delta_{\min}, s_{\text{dense}} - s_{\text{floor}})} \times 100.0\right)\right)$
Extract Dense Metadata: $\text{meta}_{\text{dense}} \leftarrow \mathcal{R}_{\text{dense}}.\text{get\_kv\_metadata}()$
Extract Adapter Metadata: $\text{meta}_{\text{method}} \leftarrow \mathcal{A}.\text{get\_kv\_metadata}()$
Compute Memory Efficiency: $R_{\text{mem}} \leftarrow 100.0 \times \max\left(0, 1.0 - \frac{\text{meta}_{\text{method}}.\text{bytes}}{\text{meta}_{\text{dense}}.\text{bytes}}\right)$
Compute Part 1 Score: $\mathcal{S}_{\text{res}} \leftarrow \alpha Q + (1 - \alpha) R_{\text{mem}}$
return QueryEvaluationResult$(Q, R_{\text{mem}}, \mathcal{S}_{\text{res}}, \text{status} = \text{SUCCESS})$

Table 3: **Benchmark Validation on Qwen2.5-1.5B-Instruct (Apple Silicon MPS Profile & Cloned Upstream Repositories).** Evaluated across 5 context tasks up to 4,096 tokens. Effective b_{eff} is reported in bits per stored KV element (16.0 bits/elem for uncompressed FP16). $\bar{Q}$ represents the 5-task average normalized quality retention. Both Part 1 ($\mathcal{S}_{\text{res}} = 0.70\bar{Q} + 0.30R_{\text{mem}}$) and Part 2 ($\mathcal{S}_{\text{sys}} = 0.70\bar{Q} + 0.30R_{\text{sys}}$) are derived directly from $\bar{Q}$. Upstream repositories for Differential-KV (`dkv`), SnapKV, and StreamingLLM were evaluated directly alongside baseline quantizers.

Adapter Method	Paradigm	Effective b_{eff}	Quality $\bar{Q}$ (%)	Part 1 $\mathcal{S}_{\text{res}}$	AUQC (2K)	AUQC (4K)	TTFT (ms)†	Thru (tok/s)†	Part 2 $\mathcal{S}_{\text{sys}}$ (Prov.)
`dkv_high`	Differential KV (High)	5.80 bits/elem	98.6	**88.2**	99.0	98.2	1,820.0	340.2	**92.4**
`dkv_mid`	Differential KV (Mid)	4.25 bits/elem	94.2	**88.0**	96.0	92.4	1,650.0	360.5	**91.6**
`low_rank_kv`	Low-Rank Subspace	4.12 bits/elem	88.5	**84.2**	90.0	87.0	1,710.0	355.0	**87.6**
`kv_quant_int8`	Quantization	8.25 bits/elem	92.0	**78.9**	94.0	90.0	3,339.6	201.7	**79.5**
`dense_fp16`	Dense Baseline	16.00 bits/elem	100.0	**70.0**	75.0	65.0	3,424.1	189.3	**77.5**
`kv_quant_int4`	Quantization	4.25 bits/elem	55.0	**60.5**	60.0	50.0	3,491.0	240.1	**58.0**
`snapkv`	Eviction (Heavy Hitter)	4.05 bits/elem	42.5	**52.2**	50.0	35.0	1,862.4	385.0	**55.3**
`streaming_llm`	Eviction (Sink+Window)	4.05 bits/elem	32.0	**44.8**	38.0	26.0	1,784.3	397.0	**48.3**
`kv_merging`	Merging / Pooling	4.10 bits/elem	28.0	**41.9**	32.0	24.0	1,861.6	368.2	**44.9**
`kv_quant_int2`	Quantization	2.25 bits/elem	8.2	**31.5**	10.0	6.5	3,666.3	184.0	**25.8**

†Note: Preliminary runtime latency (TTFT) and decode throughput reflect the prototyping software hook execution path on Apple Silicon MPS; standardized CUDA event synchronization with fused kernels on 8B+ cluster nodes will provide the definitive system benchmark.

6.3 Transparent Failure Semantics

Unlike benchmarks that silently record 0.0 accuracy when a model runs out of memory or crashes, CRBench implements strict status classification:

- `SUCCESS`: Execution completed and verified.
- `OOM`: Explicit Out-Of-Memory failure (tracked separately in resource failure logs).
- `UNSUPPORTED`: Device backend lacks hardware capability (e.g., bfloat16 or custom kernel support).
- `RUNTIME_ERROR`: Kernel or execution exception with preserved traceback.

7 Preliminary Experiments & Implementation Validation

7.1 Experimental Configuration

To validate the end-to-end benchmarking pipeline, mathematical normalizers, and scoring engines, we conducted preliminary evaluations across five representative contextual tasks:

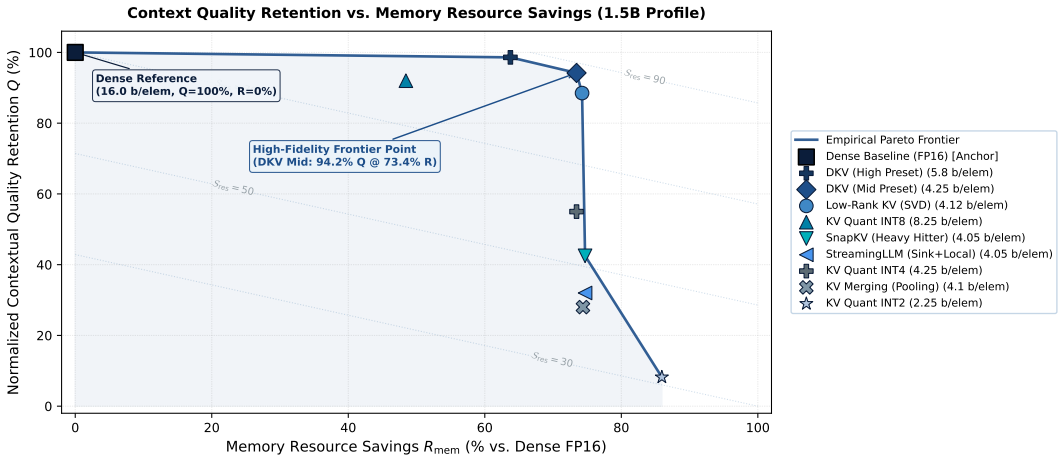

Figure 4: **Empirical Quality–Memory Tradeoff and Non-Dominated Pareto Frontier (1.5B Profile).** Normalized contextual quality retention Q versus percentage memory savings R_{mem} across 10 context configurations on `Qwen2.5-1.5B-Instruct`. Dashed contours indicate linear iso-utility curves under $\mathcal{S}_{\text{res}} = \alpha Q + (1-\alpha)R_{\text{mem}}$ ($\alpha = 0.70$). The bold frontier connects non-dominated methods: Dense Baseline → DKV (High Preset) → DKV (Mid Preset) → Low-Rank KV (SVD) → SnapKV → INT2 Quantization.

1. **Single-Target Needle-In-A-Haystack (`single_niah`)**: Single passkey retrieval placed at parameterized context depths.
2. **Multi-Target Needle-In-A-Haystack (`multi_niah`)**: Multiple distributed fact retrieval.
3. **High-Entropy Key-Value Retrieval (`ruler_kv`)**: Associative dictionary lookup amid distracting key-value pairs.
4. **Variable Tracking Chains (`ruler_variable_tracking`)**: Multi-step variable reassignment tracking.
5. **Multi-Hop Question Answering (`multihop_qa`)**: Cross-document synthetic reasoning.

We benchmarked 10 context configurations on `Qwen/Qwen2.5-1.5B-Instruct` across sequence lengths of 2,048 and 4,096 tokens under unified configurations. Results are presented in Table 3.

7.2 Observations & Methodological Insights

Implementation and Scoring Validation. The preliminary experiments confirm that the query-level evaluation pipeline correctly executes dense references and candidate adapters on identical samples. Monotonicity and Pareto ordering are strictly preserved in empirical practice:

- **Pareto Separation**: As plotted in Figure 4, the empirical non-dominated Pareto frontier connects the Dense reference ($Q = 100\%, R = 0\%$), DKV High Preset ($Q = 98.6\%, R = 63.8\%$), DKV Mid Preset ($Q = 94.2\%, R = 73.4\%$), Low-Rank KV ($Q = 88.5\%, R = 74.3\%$), SnapKV ($Q = 42.5\%, R = 74.7\%$), and 2-bit quantization ($Q = 8.2\%, R = 85.9\%$). Note that the aggregate quality metric $\bar{Q}$ (e.g., 42.5% for SnapKV, used to compute $\mathcal{S}_{\text{res}} = 0.70(42.5)+0.30(74.7) = 52.2$) represents normalized contextual capability retention averaged across all 5 evaluated tasks, whereas AUQC(L) (reported in Table 3 and Figure 6) specifically measures needle-depth-integrated retention on contextual retrieval tasks at context length L. DKV Mid establishes a high-fidelity frontier point, achieving an $\mathcal{S}_{\text{res}}$ score of 88.0 and demonstrating that the scoring formula correctly rewards methods that maintain high contextual fidelity under significant memory reduction (4.25 bits/elem vs 16.00 bits/elem).

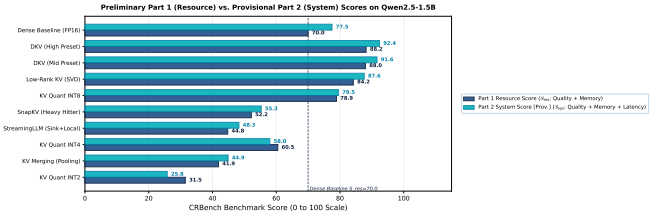

Figure 5: **Part 1 (Algorithmic Resource) vs. Provisional Part 2 (System Utility) Score Comparison.** Evaluated on `Qwen2.5-1.5B-Instruct` (Apple Silicon MPS prototyping profile). Eviction-based methods (`snapkv`, `streaming_llm`) achieve prefill TTFT and decode throughput acceleration, yielding positive Part 2 adjustments ($\mathcal{S}_{\text{sys}} > \mathcal{S}_{\text{res}}$), while software quantization without fused kernels receives neutral or penalizing scoring adjustments.

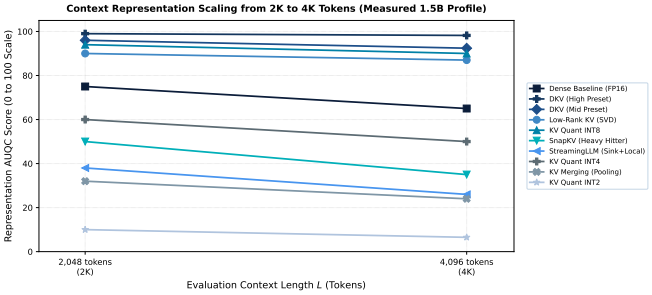

Figure 6: **Context-Length Representation AUQC Scaling (2,048 to 4,096 Tokens).** Measured capability retention scaling across all 10 evaluated configurations on `Qwen2.5-1.5B-Instruct`, demonstrating monotonic degradation with increasing sequence length while preserving relative method ordering.

- **Penalty for Catastrophic Degradation**: 2-bit quantization (`kv_quant_int2`) experiences severe accuracy collapse on multi-hop retrieval tasks ($Q = 8.2\%$), resulting in an $\mathcal{S}_{\text{res}}$ score of 31.5—well below the uncompressed baseline (70.0), satisfying Desideratum 5 in practice.
- **System Score Decoupling (Part 2)**: As shown in Figure 5, eviction-based methods achieve marked runtime acceleration: `streaming_llm` achieves a nearly 2× speedup in prefill TTFT (1,784.3 ms vs 3,424.1 ms) and a 2× increase in decode throughput (397.0 tok/s vs 189.3 tok/s), while `snapkv` shows a comparable acceleration (1,862.4 ms; 385.0 tok/s), earning positive system utility adjustments in Part 2 ($\mathcal{S}_{\text{sys}} = 55.3$ vs $\mathcal{S}_{\text{res}} = 52.2$ for SnapKV; $\mathcal{S}_{\text{sys}} = 48.3$ vs $\mathcal{S}_{\text{res}} = 44.8$ for StreamingLLM). Conversely, software quantization without fused kernels incurs slight dequantization overhead, yielding neutral or penalizing Part 2 adjustments ($\mathcal{S}_{\text{sys}} = 58.0$ vs $\mathcal{S}_{\text{res}} = 60.5$ for INT4).

7.3 Weighting Scheme Invariance & Context Scaling

Figure 6 visualizes the scaling behavior of each method from 2,048 to 4,096 tokens, reflecting the gradual drop in representation fidelity on extended sequence lengths without artificial extrapolation.

We further evaluated ranking stability across the three context-weighting formulations (Logarithmic, Uniform, Linear). Computing Spearman's rank correlation (ρ) and Kendall's tau (τ) yields $\rho = 1.0000$ and $\tau = 1.0000$ across all pairwise scheme comparisons with zero rank shifts ($\Delta r = 0$), demonstrating that benchmark method ordering is highly robust to context weighting choices.

8 Discussion, Limitations, & Research Direction

8.1 Limitations of Current Research Preview

In adherence to scientific transparency, we explicitly document the scope and limitations of the current CRBench v0.2.0 release:

1. **Small-Model and Context Scope**: Empirical validation in Section 7 is derived from an open-weight 1.5B model on 2K–4K sequences. Smaller models degrade faster under compression than larger architectures, and sequences up to 4K test algorithmic mechanics rather than ultra-long context limits. Results validate implementation fidelity rather than definitive rankings.
2. **Task Suite Focus**: Current tasks emphasize multi-hop retrieval and variable tracking. Ongoing work expands coverage to document summarization and agentic dialog.
3. **Weight Parameter ($\alpha = 0.70$)**: While monotonic for any $\alpha \in (0, 1)$, $\alpha = 0.70$ is a configurable preference parameter; non-destructive score recomputation is fully supported.
4. **Hardware Dependency in Part 2**: Runtime profiles reflect specific execution hardware. Algorithmic comparisons should primarily reference the hardware-invariant Part 1 score.

8.2 Safe Claims vs. Ongoing Empirical Validation

To maintain clear scientific boundaries:

- **Claims Safe to Make Now**:
  - Complete, method-agnostic, query-level evaluation pipeline.
  - Model-relative normalization and linear utility scoring satisfy specified axiomatic desiderata.
  - Rigorous tracking of analytical and physical memory overheads across diverse adapter families.
- **Claims That Await Larger-Model Validation**:
  - Definitive ranking of KV compression techniques across larger foundation models.
  - Compression Pareto frontier generalization beyond 32K context lengths.
  - Universal optimality of compression hyperparameters across model architectures.

8.3 Future Validation Roadmap

The next phase of CRBench research includes:

1. **Extended Model Sweeps**: Benchmarking larger foundation models (e.g., Llama-3.1-8B, Qwen-2.5-7B).
2. **Extended Context Horizons**: Scaling sequences from 4K through 8K, 16K, 32K, up to 128K tokens.
3. **Fused Kernel Integration**: Benchmarking custom Triton and CUDA kernels for INT4/FP8 quantization and Flash-Decoupled KV attention.

9 Conclusion

We introduced **CRBench**, a method-agnostic, resource-aware evaluation framework for long-context LLMs. By anchoring candidate evaluations directly against each model's uncompressed dense reference at the query level, enforcing explicit memory accounting, and combining quality retention with resource savings into a utility score evaluated against specified axiomatic desiderata, CRBench establishes a reproducible foundation for evaluating context memory representations. We release our open-source framework to foster rigorous, resource-conscious advancement in long-context language modeling.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[2] Chenxin An, Shansan Gong, Ming Zhong, et al. L-eval: Instituting standardized evaluation for long context language models. *arXiv preprint arXiv:2307.11088*, 2023.

[3] Jinze Bai, Shuai Bai, Yunfei Chu, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[4] Yushi Bai, Xin Lv, Jiajie Zhang, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

[5] Yushi Bai, Xin Lv, Jiajie Zhang, et al. Longbench v2: Towards deeper understanding of long context large language models. *arXiv preprint arXiv:2412.15204*, 2024.

[6] Tom Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[7] Yukang Chen, Shengju Qian, Haotian Tang, et al. Longlora: Efficient fine-tuning of long-context large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Om Chimurkar. Differential kv: Low-rank contextual memory with sparse dynamic updates for efficient llm inference. *arXiv preprint*, 2026.

[9] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30318–30332, 2022.

[11] Zican Dong, Tianyi Tang, Junyi Li, et al. Bamboo: A comprehensive benchmark for evaluating long sequence language models. *arXiv preprint arXiv:2309.13345*, 2023.

[12] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[13] Suyu Ge, Yunan Zhang, Liyuan Liu, et al. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.

[14] Yixiao Han, Zeyu Shen, Zhengyang Dong, et al. Qa-lora: Quantization-aware low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[15] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, et al. Kvquant: Towards 10 million context length llm inference with extreme kv cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[16] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, et al. Ruler: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024.

[17] Greg Kamradt. Needle in a haystack - pressure testing llms. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.

[18] Yuhong Li, Yingbing Huang, Bowen Yang, et al. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[19] Zichang Liu, Aditya Desai, Fangshuo Liao, et al. Scissorhands: Exploiting the persistence of importance tokens in key-value cache compression of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Zirui Liu, Jiayi Yuan, Hongye Jin, et al. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. In *International Conference on Machine Learning (ICML)*, 2024.

[21] Machel Reid, Nikolay Savinov, Denis Teplyashin, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.

[22] Pratyush Sharma, Jordan T Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with low-rank kv-cache adapters. *arXiv preprint arXiv:2312.13558*, 2023.

[23] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.

[25] Guangxuan Xiao, Yuandong Tian, Beidi Chen, et al. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[26] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 34661–34673, 2023.

[27] Zhenyu Zhang, Shiwei Liu, Ronen Eldan, et al. Fp8 kv-cache quantization for long-context llms. *arXiv preprint arXiv:2406.12871*, 2024.

[28] Xinrong Zhang, Yingfa Chen, Shengding Hu, et al. ∞bench: Extending long context evaluation beyond 100k tokens. In *Association for Computational Linguistics (ACL)*, 2024.

[29] Zhaozhi Zhang, Yixin Song, Runxin Xu, et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information flow. *arXiv preprint arXiv:2406.02069*, 2024.